Compresr SDK

Core Usage

Query-aware LLM context compression for Python and TypeScript. Drop **30–70% of the tokens** you send to GPT, Claude, or Gemini without losing the answer your user actually asked for.

1 Why Compresr

Most LLM context is wasted. Retrieved snippets, tool outputs, and chat history carry far more text than the model needs to answer the question in front of it. Compresr reads your context *together with the query* and returns a shorter version that keeps the answer-bearing content and trims the rest. Same passage, two different questions, two different compressions — and 30–70% fewer tokens billed by your model provider.

One cloud API, two thin SDKs. Same on-the-wire contract. Call `compress(...)` and ship.

2 Install & quick start

Python 3.9+ and Node 20+ are the only requirements. Grab an API key (prefix `cmp_`) from compresr.ai.

```
pip install compresr             # Python
npm install @compresr/sdk        # TypeScript
export COMPRESR_API_KEY=cmp_...
```

Python

```python
from compresr import CompressionClient, RateLimitError

client = CompressionClient()             # picks up COMPRESR_API_KEY
try:
    result = client.compress(
        context="Long passage to compress...",
        query="What is the main conclusion?",
        target_compression_ratio=0.5,    # drop ~50% of tokens
    )
except RateLimitError:
    result = None                        # back off, retry, or fall through

if result and result.success:
    print(result.data.tokens_saved, result.data.compressed_context)
else:
    print("compression skipped:", getattr(result, "error", "n/a"))
```

TypeScript

```typescript
import { CompressionClient } from '@compresr/sdk';

const client = new CompressionClient();   // picks up COMPRESR_API_KEY
const result = await client.compress({
  context: 'Long passage to compress...',
  query: 'What is the main conclusion?',
  targetCompressionRatio: 0.5,
});
if (result.success) {
  console.log(result.data.tokens_saved, result.data.compressed_context);
} else {
  console.warn('compression skipped:', result.error);
}
```

The response carries the compressed text, original and compressed token counts, achieved ratio, and call duration — everything you need to log savings and prove ROI to your finance team. Typed exceptions (`RateLimitError`, `ModelNotFoundError`, ...) let you branch on intent, and transient `429` / `503` responses retry automatically with jittered backoff.

3 Batch and async

Compress up to **100 contexts per call** for RAG indexing, replay jobs, or batch evals. Use one shared query, or pair each context with its own. Python is sync-first with full async equivalents; TypeScript is async-only.

Python

```python
# One shared query
batch = client.compress_batch(
    contexts=["Doc 1...", "Doc 2..."],
    queries="What is self-attention?",
)
# Or per-context queries
batch = client.compress_batch(inputs=[
    {"context": "Doc A...", "query": "What is X?"},
    {"context": "Doc B...", "query": "What is Y?"},
])

# Async on the same client
async with CompressionClient(api_key="cmp_...") as c:
    result = await c.compress_async(context="...", query="...")
```

TypeScript

```typescript
const batch = await client.compressBatch({
  contexts: ['Doc 1...', 'Doc 2...'],
  queries: 'What is self-attention?',
});
```

```ts
const batch2 = await client.compressBatch({ inputs: [
  { context: 'Doc A...', query: 'What is X?' },
  { context: 'Doc B...', query: 'What is Y?' },
]});
```

Streaming (`compress_stream` / `compressStream`) is available on both SDKs when you want to render compressed output as it arrives.

4 Key parameters

Every compress call takes the same knobs. Defaults are filled by the backend so you can start with two arguments and tune later.

Parameter	Default	What it does
`context`	required	The text you want shorter.
`query`	`None`	The question your LLM is answering. Drives what's kept vs. dropped.
`target_compression_ratio` (TS: `targetCompressionRatio`)	backend default	`0-1` = fraction of tokens to remove (`0.5` drops ∼50%). `>1` = Nx factor (`4` = ∼4× smaller).
`coarse`	backend default	`true` = paragraph-level (faster); `false` = token-level (finer, slower).
`compression_model_name` (TS: `compressionModelName`)	`"latte_v1"`	Which model to use (see below).
`heuristic_chunking` (TS: `heuristicChunking`)	`None`	Structure-aware chunking — useful for Markdown, code, legal docs.
`disable_placeholders` (TS: `disablePlaceholders`)	`None`	Drop placeholder markers (`<...>`) from removed spans.

5 Choose a model

- `latte_v1` — the default. Best quality on long-form prose, RAG snippets, and tool outputs.
- `latte_v2` — faster variant. Lower latency for high-throughput agent loops and tight P99 budgets.
- **Agentic variants** (`hcc_` , `toc_` , `tdc_` prefixed) target chat history, tool outputs, and tool discovery respectively — see the framework PDFs below.

Note

Quick rule: start with defaults (`latte_v1` , ratio `0.5`). Drop to `latte_v2` if you need lower latency. Tune `target_compression_ratio` based on your eval scores.

6 Pick the right framework integration

Already on a framework? Skip the plumbing — each integration has its own short PDF.

- **LangChain** — drop-in document compressor and retriever wrapper. (`python_langchain` , `typescript_langchain`)
- **LangGraph** — long-running agent memory compression for stores, checkpoints, and handoffs. (`python_langgraph` , `typescript_langgraph`)
- **LlamaIndex** — node postprocessor, chat memory, and query-engine wrappers. (`python_llamaindex` , `typescript_llamaindex`)
- **LiteLLM proxy** — server-side guardrail that compresses every request flowing through your proxy. (`python_litellm`)

Questions, pricing, or a custom deployment? Reach us at compresr.ai or founders@compresr.ai.